Assignment 5: Instructional Fine-Tuning of LLaMA for NER

Executive Summary

This assignment implements and compares **three** instructional fine-tuning strategies for Named Entity Recognition (NER) on CoNLL-2003, using a LLaMA-based large language model:

1. **Full Fine-Tuning** (required) — all 1.1 B parameters updated
2. **LoRA** (optional) — only ~4.5 M adapter parameters updated; base weights frozen in fp16
3. **QLoRA** (optional) — same adapters, but base weights frozen in 4-bit NF4 quantization

Model: `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (identical architecture to LLaMA 2, freely accessible).
Notebook: `Assignment5_Instructional_Fine-Tuning/MingHsiangLee_Assignment5.ipynb`
Environment: Google Colab, T4 GPU (15.6 GB VRAM), Python 3.10

---

1. Dataset

CoNLL-2003

Split	File	Sentences
Train	eng.train	14,041
Validation	eng.testa	3,250
Test	eng.testb	3,453

Entity types: `PER` (person), `ORG` (organization), `LOC` (location), `MISC` (miscellaneous) Tag scheme: BIO

Fast-mode limits used in this run: train = 500, val = 100, test = 100. Full-mode training (all splits) is supported by setting `FAST_MODE = False` in the notebook.

Note on data split: The standard CoNLL-2003 community split is 68% / 16% / 16% (14,041 / 3,250 / 3,453), not the ideal 80/10/10. This split was retained rather than re-partitioned for two reasons: (1) it is the universally accepted benchmark split, enabling direct comparison with published results; (2) re-shuffling across the existing splits would contaminate the standard evaluation set and produce non-comparable metrics. The validation and test fractions (~16% each) exceed the 10% requirement, providing more evaluation signal than strictly required.

---

2. Model Selection

TinyLlama/TinyLlama-1.1B-Chat-v1.0

- Architecture: identical to LLaMA 2 (grouped-query attention, SwiGLU, RoPE)
- Parameters: 1.1 B
- Pre-training: 3 T tokens from SlimPajama and Starcoderdata

- Instruction tuning: UltraChat + UltraFeedback (chat-formatted)
- HuggingFace accessibility: no gating / no license agreement required

The model was chosen because it uses the LLaMA 2 architecture, fits in T4 VRAM under all three fine-tuning conditions, and is freely accessible without an HuggingFace login.

> Alternative: `meta-llama/Llama-3.2-1B-Instruct` can be substituted by changing `MODEL_NAME` in the configuration cell (requires HF account and model-access agreement).

---

3. Instructional Format

Each CoNLL-2003 sentence is formatted as a TinyLlama chat prompt:

```
<|system|>
You are a named entity recognition (NER) expert. Extract all named
entities ...
Return [] if no named entities are present. Output ONLY the JSON array.
</s>
<|user|>
Text: EU rejects German call to boycott British lamb .</s>
<|assistant|>
[{"entity": "EU", "type": "ORG"}, {"entity": "German", "type": "MISC"},
{"entity": "British", "type": "MISC"}]</s>
```

Training: the full prompt (system + user + assistant answer) is tokenized. Instruction tokens are masked to `-100`; loss is computed **only on the JSON output tokens**.

Inference: only the system + user prefix is fed to the model; the model generates the assistant turn (JSON array) autoregressively.

Entity alignment for evaluation: Predicted entity dicts are re-aligned to the original BIO token sequence using case-insensitive exact token-span matching (`entities_to_bio`), then evaluated with `seqeval`.

---

4. Hyperparameters

Shared

Hyperparameter	Value
Max input sequence length	256 tokens
Max new tokens (inference)	80 tokens
Train batch size (per device)	4
Gradient accumulation steps	4 → effective batch = 16
Epochs	3
Warmup steps	50

Hyperparameter	Value
Weight decay	0.01
Precision	fp16 AMP (Colab T4)
Gradient checkpointing	enabled

Full Fine-Tuning specific

Hyperparameter	Value
Optimizer	adamw_8bit (bitsandbytes)
Learning rate	2e-5
Base model dtype	float32
Trainable parameters	1,100,048,384 (100%)

LoRA specific

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1e-4
LoRA rank r	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, v_proj, k_proj, o_proj
Trainable parameters	4,505,600 (~0.41%)

QLoRA specific

Hyperparameter	Value
Optimizer	paged_adamw_8bit (bitsandbytes)
Learning rate	1e-4
Base model quantization	4-bit NF4 + double quantization
Compute dtype	float16
LoRA rank r	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	q_proj, v_proj, k_proj, o_proj

Hyperparameter	Value
Trainable parameters	4,505,600 (~0.41%)

QLoRA workflow:

1. Load base model in 4-bit via `BitsAndBytesConfig` (NF4, double quantization enabled)
2. Call `prepare_model_for_kbit_training()` — casts layer norms to fp32, enables gradient checkpointing
3. Attach LoRA adapters with `get_peft_model()` — same config as LoRA experiment
4. Train only adapter weights; base model stays quantized and frozen throughout

---

5. Evaluation Protocol

1. **Generation:** for each test sentence, the instruction prefix is fed to the fine-tuned model and up to 80 new tokens are generated greedily (`do_sample=False`).
2. **Parsing:** the generated text is parsed as JSON (`json.loads`) with a regex fallback to extract the first `[...]` array found in the output.
3. **Alignment:** predicted entity dicts are mapped back to the token sequence via case-insensitive token-span matching.
4. **Metrics (seqeval, entity-level):**
   - Precision, Recall, F1-score
   - Per-class breakdown (PER, ORG, LOC, MISC)
5. **Resource metrics:**
   - Training wall-clock time (seconds)
   - Peak GPU memory (MB, `torch.cuda.max_memory_allocated`)
   - CPU RAM (psutil)
   - JSON parse success rate

---

6. Results

6.1 Resource Usage and Training

Metric	Full Fine-Tuning	LoRA	QLoRA
Trainable parameters	1,100 M (100%)	4.5 M (0.41%)	4.5 M (0.41%)
Base model precision	float32	fp16	4-bit NF4
GPU peak memory (MB)	14,055	9,810	11,086
Training time (s)	258.4	**166.5**	241.1
Final training loss	**0.1006**	0.2229	0.2145
Speedup vs Full FT	1.0×	**1.55×**	1.07×
GPU memory vs Full FT	—	**−30%**	−21%

LoRA is the most time-efficient method (1.55× faster than Full FT). QLoRA trains slower than LoRA (~45%) because of 4-bit dequantization overhead during forward passes, and its GPU peak (11,086 MB) is higher than LoRA's (9,810 MB) because fp16 dequantization buffers must be materialized for every matrix multiply.

6.2 NER Performance (test set, 100 sentences, fast-mode run)

Metric	Full Fine-Tuning	LoRA	QLoRA
Entity Precision	**0.9286**	0.8603	0.8456
Entity Recall	**0.7189**	0.5392	0.5300
Entity F1	**0.8104**	0.6629	0.6516
JSON Parse Rate	**100%**	100%	100%
Inference time (s)	153.0	143.2	224.9

6.3 Per-class F1 (Full Fine-Tuning)

Class	Precision	Recall	F1	Support
LOC	0.95	0.98	**0.96**	82
MISC	0.68	0.77	0.72	22
ORG	1.00	0.50	0.67	2
PER	1.00	0.52	0.69	111

6.4 Per-class F1 (LoRA)

Class	Precision	Recall	F1	Support
LOC	0.98	0.66	0.79	82
MISC	0.52	0.50	0.51	22
ORG	0.00	0.00	0.00	2
PER	0.93	0.47	0.62	111

6.5 Per-class F1 (QLoRA)

Class	Precision	Recall	F1	Support
LOC	0.96	0.65	0.77	82
MISC	0.46	0.59	0.52	22
ORG	0.00	0.00	0.00	2
PER	0.98	0.44	0.61	111

LOC is the easiest class across all methods (strong geographic cues, no ambiguity). MISC remains consistently harder (ambiguous category, no strong syntactic cues). ORG has only 2 test entities in the 100-sentence fast-mode subset, making its per-class metrics unreliable — Full FT recovers both by memorizing the format, while adapter methods miss at least one. PER recall is low across all adapter methods despite high precision, suggesting the model generates precise entity spans when it does output PER, but often skips them.

6.6 Cross-Assignment Comparison

Assignment	Method	Entity F1
A2	CRF	0.7905
A2	BiLSTM	0.6347
A3	BERT (token classification)	**0.9108**
A5	TinyLlama Full Fine-Tuning	0.8104
A5	TinyLlama LoRA	0.6629
A5	TinyLlama QLoRA	0.6516

---

7. Analysis and Discussion

7.1 Accuracy ordering: Full FT > LoRA ≈ QLoRA

Full FT > LoRA: Full FT updates all 1.1 B parameters, allowing every layer to specialize for the NER task and the JSON output format. The training loss converges to 0.1006 — much lower than LoRA's 0.2229 — indicating much tighter memorization of the instructional pattern. The gap (ΔF1 ≈ 0.148) is larger than typical in the literature, likely because 500 training samples are a small fine-tuning budget: Full FT can overfit to the training format while adapter methods, with only 0.41% of parameters trainable, cannot fully absorb it.

LoRA ≈ QLoRA: The two adapter methods are nearly identical in accuracy (ΔF1 = 0.011). QLoRA's 4-bit NF4 base weights introduce a small quantization error that accumulates through the frozen layers, slightly reducing the adapter's effective gradient signal. The gap is small because NF4 quantization is highly accurate for LLM weights compared to uniform quantization.

7.2 Why All Methods Achieve 100% JSON Parse Rate

Unlike fast-mode local CPU runs (where models would sometimes output prose or repeat instructions), the Colab T4 GPU with properly calibrated sequence lengths and 3 training epochs produces reliable JSON on every test sample. This validates the instructional format design: with sufficient compute and fine-tuning, the generative NER approach is format-stable.

7.3 The Low Recall Problem for LoRA / QLoRA

Both adapter methods achieve high precision (0.86 / 0.85) but low recall (0.54 / 0.53). The model correctly labels entities it does extract, but frequently skips them — especially for PER (recall 0.47 / 0.44). This pattern suggests the adapters learn which spans are entities but do not fully learn the coverage behavior

(outputting every entity in the sentence). Full FT, with 0.1006 training loss vs 0.22+, achieves much better recall (0.72).

7.4 Why Full FT Still Falls Below BERT (A3, F1 = 0.91)

Factor	BERT token cls	TinyLlama generative
Training data (fast mode)	500 sentences	500 sentences
Output space	Fixed label per token	Free-form JSON
Task framing	Direct token labeling	Span extraction + type assignment
Failure modes	None (always produces labels)	Span miss (skipped entities)
Entity span detection	Implicit in BIO tags	Requires exact span reproduction

BERT's token classification head always produces one label per token, so recall is never zero. The generative approach must additionally reproduce the exact surface form of the entity span in the JSON output. With the full 14 k training sentences and a larger model (e.g., LLaMA 3.2 7B), the generative approach is expected to match or exceed BERT.

7.5 QLoRA Memory Behavior on T4

An unexpected finding: QLoRA's GPU peak (11,086 MB) is **higher than LoRA's** (9,810 MB), despite having a quantized base model. This occurs because:

1. The 4-bit base weights are stored compactly, but dequantized to fp16 for every forward/backward pass matrix multiply.
2. These fp16 dequantization buffers are large and live in GPU memory during training.
3. The `paged_adamw_8bit` optimizer partially offloads optimizer states, but this saving is more than offset by dequantization buffer allocation.

On a GPU with < 8 GB VRAM (e.g., free-tier Colab), QLoRA's advantage would be much more pronounced, since LoRA would OOM while QLoRA would not.

7.6 QLoRA Optimizer: paged_adamw_8bit

QLoRA uses `paged_adamw_8bit` (from bitsandbytes) instead of standard AdamW. This reduces memory by quantizing optimizer states to 8-bit and paging them to CPU when not in use. It does not change the training dynamics materially but enables more aggressive memory reduction on small GPUs.

---

8. Trade-off Summary

Dimension	Full Fine-Tuning	LoRA	QLoRA
Entity F1	**0.8104**	0.6629	0.6516
GPU peak memory	14,055 MB	**9,810 MB**	11,086 MB
Training time (s)	258.4	**166.5**	241.1

Dimension	Full Fine-Tuning	LoRA	QLoRA
Trainable params	1,100 M (100%)	4.5 M (0.41%)	4.5 M (0.41%)
Base model precision	float32	fp16	4-bit NF4
JSON parse rate	100%	100%	100%
Storage per task	Full model (~2.2 GB)	Adapter (~35 MB)	Adapter (~35 MB)
Multi-task scaling	One copy per task	Shared base + deltas	Shared base + deltas
Ease of implementation	Simple	PEFT library	PEFT + bitsandbytes

Recommendation by use case:

- **QLoRA**: resource-constrained GPU (< 8 GB VRAM), deploying many task variants from a single shared base. On T4 the memory advantage over LoRA is modest.
- **LoRA**: best balance of speed and accuracy on mid-range GPUs (8–16 GB); 36% faster than Full FT at only −0.148 F1.
- **Full FT**: maximum accuracy when GPU memory is not a constraint and highest F1 is required.

---

9. Conclusion

This assignment demonstrates instructional fine-tuning of a 1.1 B LLaMA-based model for NER across three configurations, showing a clear resource-vs-accuracy trade-off progression.

Key outcomes:

1. All three methods achieve **100% JSON parse rate** on Colab T4 GPU — the instructional format is format-stable after 3 epochs of fine-tuning.
2. **Full FT > LoRA > QLoRA** in entity F1 (0.8104 / 0.6629 / 0.6516).
3. Full FT surpasses CRF (0.79) and BiLSTM (0.63) from prior assignments, approaching BERT token classification (0.9108) despite a generative output format.
4. LoRA is **36% faster** and uses **30% less GPU memory** than Full FT, at a cost of −0.148 F1.
5. QLoRA's memory advantage over LoRA is smaller than expected on T4 (11,086 vs 9,810 MB) due to fp16 dequantization buffers; its advantage is more pronounced on < 8 GB GPUs.
6. Low recall (not low precision) is the dominant failure mode for adapter methods — the model identifies entity spans correctly when it outputs them, but frequently omits entities.
7. The adapter-only approach (LoRA / QLoRA) makes deploying many task-specific model variants from a single shared base highly practical and storage-efficient (~35 MB adapter vs ~2.2 GB full model copy per task).